

DIRECTORIO DE PERSONAS

Explicacion Completa delCodigo

Para **principiantes totales** - Sin asumir ningun conocimiento previo
Vite + React 18 | PRG4 - Duin | Carpeta "2- Personas"

Que vas a encontrar

Explicacion linea por linea de cada archivo | Analogias con LEGO, cocina y vida real | Diagramas de flujo | Glosario y preguntas frecuentes |
11 capitulos - 0% jerga sin explicar

Septiembre 2026 | Documento generado automaticamente a partir del codigo fuente

100% fiel al repositorio - Cada linea verificada

Explicación Completa del Proyecto "Personas" — Para Principiantes Totales

Curso: PRG4 - Duin / Carpeta: 2- Personas

Nivel: Cero absoluto. Si nunca programaste, este documento es para ti.

Objetivo: Entender línea por línea qué hace cada archivo y por qué existe.

Índice

- 1 La idea en una frase
- 2 Analogía general: ¿Qué es esta app?
- 3 Tecnologías usadas (explicadas como si tuvieras 10 años)
- 4 Estructura de carpetas: el mapa del proyecto
- 5 Cómo poner en marcha el proyecto (paso a paso)
- 6 Explicación detallada archivo por archivo
- 7 Conceptos clave de programación explicados para inexpertos
- 8 Flujo completo de datos: qué pasa desde que abres la página
- 9 Glosario para no perderse
- 10 Errores comunes y preguntas de principiante
- 11 Qué se podría mejorar

1. La idea en una frase

La aplicación "Personas" es un directorio de 50 personas inventadas que se descargan de Internet. Puedes buscarlas por nombre y marcar tus favoritas con un clic.

Si fuera el mundo real, imagina una lista de contactos de tu teléfono: ves nombre, edad, email y SSN (un número de identificación de EE.UU.). Haces clic en una tarjeta y se pone verde = está seleccionada.

2. Analogía general: ¿Qué es esta app?

Piensa en **LEGO**:

- **React** son los bloques de LEGO. Cada bloque es un pedazo de la página (una tarjeta, un buscador, una grilla).
- **Vite** es la mesa donde armas los LEGO. Es una herramienta que te ayuda a construir y probar rápido. Sin Vite, armar LEGO sería lentísimo.
- **JavaScript** es el idioma con el que le dices a los bloques qué hacer ("si me hacen clic, pónganse verdes").
- **La API** `generate-random.org` es una fábrica externa que te manda 50 muñecos LEGO ya hechos (personas falsas) para que no tengas que crearlos a mano.

La app vive en el navegador. No guarda nada en base de datos: todo está en la memoria mientras la pestaña está abierta.

3. Tecnologías usadas

Tecnología	Qué es en palabras simples	Para qué se usa aquí
HTML (<code>index.html</code>)	El esqueleto de la página. Dice "aquí va el contenido".	Crea un <code><div id="root"></code> vacío donde React va a inyectar todo.
CSS (<code>index.css</code> , <code>App.css</code>)	La ropa y el maquillaje. Colores, tamaños, centrado.	Hace que la página no se vea fea, centra el contenido, da colores a botones.
JavaScript	El cerebro. Hace que las cosas se muevan.	Lógica de búsqueda, selección, traer datos.
React 18.3.1	Una biblioteca de JavaScript para construir interfaces con piezas reutilizables.	Todos los archivos <code>.jsx</code> son piezas React.
JSX	HTML dentro de JavaScript. Te permite escribir <code><h1>Hola</h1></code> dentro de código.	Se ve en <code>App.jsx</code> , <code>Personas.jsx</code> , etc.
Vite 5.4.10	Herramienta que arranca un servidor de desarrollo ultra rápido y empaqueta la app para producción.	Comandos <code>npm run dev</code> , <code>npm run build</code> .
Node.js + npm	El "sistema operativo" para programar en JavaScript fuera del navegador. <code>npm</code> es la tienda de apps.	Instalar React, Vite, etc.
ESLint	Un corrector ortográfico pero para código. Te avisa si escribiste algo mal.	<code>eslint.config.js</code>
API generate-random.org	Un servicio web que genera personas falsas (nombre, apellido, email, ssn, edad).	<code>fetch("https://generate-random.org/api/v1/generate/persons?locale=en_US&count=50")</code>

¿Node, Vite, React, todo eso es distinto? Sí. Imagina que quieres hacer una pizza:

* **Node/npm** = La cocina y el supermercado.

* **Vite** = El horno rápido.

* **React** = La receta de la pizza.

* **Tu código** = Los ingredientes que tú pones.

4. Estructura de carpetas

```

2- Personas/
■■■ Personas/      <-- Proyecto real (aquí está todo)
  ■■■ .vs/         <-- Configuración de Visual Studio (ignorable)
  ■■■ .gitignore   <-- Lista de archivos que Git debe ignorar
  ■■■ components/  <-- Piezas pequeñas y reutilizables
    ■ ■■■ PersonasFiltro.jsx <-- Barra de búsqueda + checkbox
    ■ ■■■ PersonasGrid.jsx  <-- Grilla de 3 columnas que acomoda tarjetas
    ■ ■■■ PersonasList.jsx  <-- UNA tarjeta de UNA persona (aunque se llame "List")
  ■■■ page/        <-- "Páginas" o vistas grandes
    ■ ■■■ Personas.jsx     <-- CEREBRO de la app: trae datos, filtra, guarda estado
  ■■■ public/
    ■ ■■■ vite.svg        <-- Iconito
  ■■■ src/            <-- Código fuente principal
    ■ ■■■ App.jsx         <-- Componente raíz que une todo
    ■ ■■■ App.css         <-- Estilos del componente App (casi no se usa)
    ■ ■■■ index.css       <-- Estilos globales de toda la página
    ■ ■■■ main.jsx        <-- Punto de entrada: le dice al navegador "empieza por App"
    ■ ■■■ assets/         <-- Imágenes (vacío en este proyecto)
  ■■■ eslint.config.js   <-- Reglas del corrector de código
  ■■■ index.html         <-- ÚNICO archivo HTML. Todo lo demás se inyecta aquí
  ■■■ package.json       <-- "DNI" del proyecto: nombre, dependencias, scripts
  ■■■ package-lock.json  <-- Candado de versiones exactas (no tocar)
  ■■■ vite.config.js     <-- Configuración del horno Vite

```

Regla de oro para principiantes: Si te pierdes, sigue este orden de lectura: `package.json` → `index.html` → `src/main.jsx` → `src/App.jsx` → `page/Personas.jsx` → `components/*`

5. Cómo poner en marcha el proyecto

Paso a paso, como si fuera la primera vez:

- 1 **Instalar Node.js** desde <https://nodejs.org> (versión LTS). Esto te da `node` y `npm`.
- 2 Abrir una terminal (PowerShell o CMD) **dentro** de la carpeta `Personas/Personas`.
- 3 Escribir:

```
npm install
```

Lenguaje: bash

Esto lee `package.json` y descarga React, Vite, etc. en una carpeta `node_modules` (puede tardar 1 minuto).

- 1 Luego:

```
npm run dev
```

Lenguaje: bash

Vite te dirá algo como Local: `http://localhost:5173`. Abre ese link en Chrome.

- 1 **Para compilar para producción:**

```
npm run build  
npm run preview
```

Lenguaje: bash

- 1 **Para revisar errores de estilo:**

```
npm run lint
```

Lenguaje: bash

6. Explicación detallada archivo por archivo

6.1 `package.json` — El DNI del proyecto

```
{  
  "name": "personas",  
  "private": true,  
  "version": "0.0.0",  
  "type": "module",  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
    "lint": "eslint .",  
    "preview": "vite preview"  
  },  
  "dependencies": {  
    "react": "^18.3.1",  
    "react-dom": "^18.3.1"  
  },  
  "devDependencies": { ... }  
}
```

Lenguaje: json

- **Línea por línea:**
- `"name": "personas"` → Nombre del proyecto. Si lo subieras a npm, se llamaría así.
- `"private": true` → Evita que lo publiques por accidente.
- `"type": "module"` → Nos permite usar `import / export` modernos en lugar de `require()`.
- `"scripts"` → Atajos. En vez de escribir `npm run dev`, escribes `dev`. Son 4 atajos: `dev` (desarrollo), `build` (empaquetar), `preview` (ver el empaquetado), `lint` (corregir).
- `"dependencies"` → Librerías necesarias **cuando la app está funcionando** en el navegador. Aquí: `react` (el motor) y `react-dom` (el conector entre React y el HTML).
- `"devDependencies"` → Librerías solo necesarias **mientras programas**: Vite, ESLint, plugins. No se envían al usuario final.

6.2 index.html — El único HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Lenguaje: html

- **¿Por qué solo hay un div?** Porque React es como un pintor que entra a una casa vacía (`<div id="root">`) y pinta todas las paredes. Toda la interfaz se genera con JavaScript y se mete dentro de ese div.
- `<script type="module" src="/src/main.jsx">` → Le dice al navegador: "Ejecuta este archivo JavaScript, que es el punto de partida".
- `lang="en"`, `charset`, `viewport` → Detalles técnicos para que el navegador sepa el idioma, los caracteres y que se adapte a móviles.

6.3 vite.config.js — Configuración del horno Vite

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'

export default defineConfig({
  plugins: [react()],
})
```

Lenguaje: js

- **7 líneas que hacen magia:** Le dice a Vite: "Usa el plugin de React con SWC". SWC es un traductor ultra rápido que convierte JSX en JavaScript que el navegador entiende, y además habilita "Fast Refresh" (cuando guardas un archivo, el navegador se actualiza sin perder el estado).
- Sin este archivo, Vite no sabría que es un proyecto React.

6.4 src/main.jsx — La puerta de entrada

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Lenguaje: js

- **Línea 1:** Trae `StrictMode`, una herramienta de React que te avisa en consola si haces algo mal. No afecta al usuario.
- **Línea 2:** Trae `createRoot`, la función que conecta React con el `div id="root"` del HTML.
- **Línea 3:** Importa los estilos globales. En React, importar un CSS significa "aplica estos estilos".
- **Línea 4:** Importa el componente principal `App`.
- **Líneas 6-10:** Busca el elemento con id `root` y **renderiza** (dibuja) `<App />` dentro. `StrictMode` envuelve a `App` para hacer chequeos extra en desarrollo.

Analogía: `main.jsx` es el portero del edificio que dice: "Todo empieza en `App`".

6.5 src/App.jsx — El contenedor principal

```
import { useState } from 'react'
import Personas from '../page/Personas';

export default function App() {
  return (
    <div style={{ maxWidth: '1000px', margin: '0 auto', padding: '20px', fontFamily: 'sans-serif' }}>
      <header style={{ borderBottom: '1px solid #ccc', marginBottom: '20px', paddingBottom: 10px }}>
        <h1>Directorio de Usuarios</h1>
      </header>
      <main>
        <Personas />
      </main>
    </div>
  );
}
```

Lenguaje: js

- `import { useState } →` Se importa pero **no se usa** en este archivo. Es código sobrante (podría borrarse). No rompe nada, solo es innecesario.
- `import Personas from '../page/Personas' →` `../` significa "salir de la carpeta `src` y entrar a `page`". Trae el componente `Personas`.
- `export default function App() →` Define el componente `App`. Un componente es una función que devuelve HTML (JSX).
- `<div style={{...}}> →` Un contenedor con estilo en línea (inline). `maxWidth: '1000px'` limita el ancho, `margin: '0 auto'` lo centra, `padding` da aire.
- `<header> →` Una cabecera con título "Directorio de Usuarios" y una línea gris debajo.
- `<Personas /> →` Aquí se incrusta toda la lógica de la app. `App` no hace mucho; delega todo a `Personas`.

6.6 page/Personas.jsx — EL CEREBRO (el archivo más importante)

Este archivo tiene **85 líneas** y hace 4 cosas: traer datos, guardar estado, filtrar y renderizar.

Imports y estados

```
import { useState, useEffect } from 'react';
import PersonasFiltro from '../components/PersonasFiltro';
import PersonasGrid from '../components/PersonasGrid';

export default function Personas() {
  const [personas, setPersonas] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [busqueda, setBusqueda] = useState("");
  const [mostrarSeleccionados, setMostrarSeleccionados] = useState(false);
```

Lenguaje: js

- `useState` → Un "gancho" (hook) de React para guardar datos que cambian. Piensa en una caja con etiqueta.
- `personas` → Array con las 50 personas. Empieza vacío `[]`.
- `cargando` → Booleano que dice si aún estamos esperando a la API. Empieza en `true`.
- `busqueda` → Texto que escribe el usuario en el buscador. Empieza vacío `""`.
- `mostrarSeleccionados` → Booleano del checkbox. Empieza en `false`.
- Cada `useState` devuelve dos cosas: el valor actual y una función para cambiarlo (ej: `personas` y `setPersonas`).

useEffect + fetch (traer datos de Internet)

```
useEffect(() => {
  async function fetchData() {
    try {
      const respuesta = await fetch("https://generate-random.org/api/v1/generate/persons?loc");
      const data = await respuesta.json();
      console.log("Respuesta de la API:", data);
      const personasDeApi = data.data;
      console.log("Personas procesadas:", personasDeApi);
      const personasConEstado = personasDeApi.map(persona => ({
        ...persona,
        Seleccionado: "n"
      }));
      setPersonas(personasConEstado);
    } catch (error) {
      console.error("Error al traer los datos:", error);
    } finally {
      setCargando(false);
    }
  }
  fetchData();
}, []);
```

Lenguaje: js

- **Desglose para inexpertos:**
- `useEffect(() => {...}, [])` → Significa: "Ejecuta este código **una sola vez** cuando el componente aparece en pantalla". El `[]` vacío es la clave: sin dependencias, solo al inicio.
- `async function fetchData()` → Una función asíncrona. "Asíncrono" = "voy a esperar a que Internet responda sin congelar la página".
- `await fetch(...)` → `fetch` es como mandar una carta y esperar respuesta. `await` = "espera aquí hasta que llegue la respuesta". La URL pide 50 personas en inglés USA.
- `await respuesta.json()` → La respuesta viene en formato JSON (texto). `json()` lo convierte en objeto JavaScript.
- `data.data` → La API devuelve algo como `{ data: [ {...}, {...} ] }`. Nos interesa el array interno.

- `.map(persona => ({ ...persona, Seleccionado: "n" })))` → Por cada persona, **copia** todos sus datos (`...persona`) y agrega un campo nuevo `Seleccionado` con valor `"n"` (no seleccionado). `map` es como pasar por una fábrica y agregarle un sticker a cada objeto.
- `setPersonas(personasConEstado)` → Guarda el array modificado en el estado. Esto hace que React vuelva a dibujar la pantalla.
- `catch` → Si Internet falla o la API bloquea (CORS), atrapa el error y lo muestra.
- `finally { setCargando(false) }` → Se ejecuta siempre, con éxito o error. Apaga el cartel "Cargando..."
- `fetchData()` → Llama a la función definida arriba.

¿Qué trae cada persona? Ejemplo real:

```
```json
```

```
{ "first_name": "John", "last_name": "Doe", "age": 28, "ssn": "123-45-6789", "email": "john@example.com" }
```

```
```
```

toggleSeleccion (marcar/desmarcar)

```
const toggleSeleccion = (ssnId) => {
  setPersonas((prevPersonas) =>
    prevPersonas.map((persona) => {
      if (persona.ssn === ssnId) {
        return {
          ...persona,
          Seleccionado: persona.Seleccionado === "s" ? "n" : "s"
        };
      }
      return persona;
    })
  );
};
```

Lenguaje: js

- Recibe un `ssnId` (el DNI de la persona clicada).
- `setPersonas` con función → Forma segura de actualizar estado basado en el valor anterior.
- `map` recorre todas las personas. Si encuentra la que tiene `ssn === ssnId`, **invierte** su estado: si era `"s"` pasa a `"n"` y viceversa. Las demás se devuelven intactas.
- Usa `ssn` como identificador porque la API no da un `id` propio.

Filtrado (búsqueda + checkbox)

```
const termino = busqueda.trim().toLowerCase();

const personasAMostrar = personas.filter((persona) => {
  const coincideSeleccion = mostrarSeleccionados ? persona.Seleccionado === "s" : true;
  const nombreCompleto = `${persona.first_name} ${persona.last_name}`.toLowerCase();
  const coincideBusqueda = nombreCompleto.includes(termino);
  return coincideSeleccion && coincideBusqueda;
});
```

Lenguaje: js

- `busqueda.trim().toLowerCase()` → Quita espacios y pasa a minúsculas para comparar sin importar mayúsculas.
- `filter` → Como un colador: solo deja pasar las personas que cumplen **ambas** condiciones.
- `coincideSeleccion` → Si el checkbox está activo, solo deja pasar las seleccionadas (`"s"`). Si no, deja pasar a todas (`true`).

- `nombreCompleto` → Une nombre y apellido en minúsculas.
- `coincideBusqueda` → `includes` pregunta: "¿el nombre contiene el texto buscado?". Si `termino` es "", siempre es `true`.
- `return coincideSeleccion && coincideBusqueda` → Debe cumplir las dos a la vez.

Renderizado (lo que se ve)

```
return (
  <div>
    <h2>Directorio de Personas</h2>
    <PersonasFiltro
      busqueda={busqueda}
      setBusqueda={setBusqueda}
      mostrarSeleccionados={mostrarSeleccionados}
      setMostrarSeleccionados={setMostrarSeleccionados}
    />
    {cargando ? (
      <p>Cargando datos desde la API...</p>
    ) : (
      <PersonasGrid
        personas={personasAMostrar}
        onToggle={toggleSeleccion}
      />
    )}
  </div>
);
```

Lenguaje: js

- `<h2>` título.
- `<PersonasFiltro>` → Se le pasan 4 props: el texto, su setter, el booleano y su setter. Así el hijo puede leer y modificar el estado del padre.
- Operador ternario `{cargando ? ... : ...}` → Si `cargando` es `true`, muestra "Cargando...". Si no, muestra la grilla.
- `<PersonasGrid>` recibe la lista **ya filtrada** y la función para toggle.

Comentario final del archivo: Hay un bloque comentado (líneas 92-114) con código viejo/incompleto. Está desactivado, no afecta. Es un borrador previo que el autor dejó.

6.7 components/PersonasFiltro.jsx — El buscador

```
export default function PersonasFiltro({
  busqueda,
  setBusqueda,
  mostrarSeleccionados,
  setMostrarSeleccionados
}) {
  return (
    <div style={{ marginBottom: '20px' }}>
      <input
        type="text"
        placeholder="Buscar por nombre o apellido..."
        value={busqueda}
        onChange={(e) => setBusqueda(e.target.value)}
        style={{ padding: '8px', marginRight: '15px' }}
      />
      <label style={{ cursor: 'pointer' }}>
        <input
          type="checkbox"
          checked={mostrarSeleccionados}
          onChange={(e) => setMostrarSeleccionados(e.target.checked)}
        /> Mostrar solo seleccionados
      </label>
    </div>
  );
}
```

Lenguaje: js

- Recibe 4 props (desestructuradas). No tiene estado propio; es "tonto" y solo avisa al padre cuando algo cambia.
- `<input type="text">` → Campo de texto controlado: `value={busqueda}` lo ata al estado del padre. `onChange` se dispara en cada tecla y llama `setBusqueda` con lo nuevo.
- `<input type="checkbox">` → `checked={mostrarSeleccionados}` y `onChange` con `e.target.checked` (true/false).
- Todo con estilos en línea simples.

6.8 components/PersonasGrid.jsx — La grilla

```
import PersonasList from './PersonasList';

export default function PersonasGrid({ personas, onToggle }) {
  return (
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr 1fr', gap: '15px' }}>
      {personas.map(persona => (
        <PersonasList
          key={persona.ssn}
          persona={persona}
          onToggle={() => onToggle(persona.ssn)}
        />
      ))}
    </div>
  );
}
```

Lenguaje: js

- Recibe `personas` (array filtrado) y `onToggle` (función del abuelo `Personas`).
- `display: 'grid', gridTemplateColumns: '1fr 1fr 1fr'` → CSS Grid de 3 columnas iguales. Cada tarjeta ocupa 1 fracción.
- `personas.map` → Por cada persona, crea un componente `<PersonasList>`.
- `key={persona.ssn}` → React necesita una clave única para cada elemento de lista para no confundirse al actualizar.

- `onToggle={() => onToggle(persona.ssn)}` → Crea una función que, al ser llamada, invoca `onToggle` con el `ssn` correcto. Se pasa hacia abajo.

6.9 components/PersonasList.jsx — La tarjeta individual

```
export default function PersonasList({ persona, onToggle }) {
  const isSelected = persona.Seleccionado === "s";

  return (
    <div
      className={`card ${persona.age < 35 ? "text_rojo" : ""} ${isSelected ? "card--selected" : ""}`}
      onClick={onToggle}
      style={{
        border: isSelected ? '2px solid #28a745' : '1px solid #ccc',
        padding: '15px',
        cursor: 'pointer',
        borderRadius: '8px'
      }}
    >
      <p><strong>Persona:</strong> {persona.last_name}, {persona.first_name}</p>
      <small>Edad: {persona.age} - SSN: {persona.ssn}</small>
      <br />
      <small>Email: {persona.email}</small>
    </div>
  );
}
```

Lenguaje: js

- **Nombre confuso:** Se llama `PersonasList` pero representa **una** persona, no una lista. Un nombre más claro sería `PersonaCard`.
- `isSelected` → Booleano que vale `true` si `Seleccionado === "s"`.
- `className` dinámico → Si `age < 35` agrega `text_rojo`, si está seleccionado agrega `card--selected`. (Las clases no tienen CSS definido en este proyecto, así que no hacen efecto visual hoy; el efecto real viene del `style`).
- `onClick={onToggle}` → Al hacer clic en la tarjeta, llama a la función que invierte la selección.
- `style` condicional → Borde verde grueso (`#28a745`) si seleccionado, gris fino si no.
- Muestra: Apellido, Nombre / Edad - SSN / Email. Usa `persona.last_name`, `first_name`, `age`, `ssn`, `email` que vienen de la API.

6.10 src/index.css y src/App.css — Los estilos

- `index.css` → Estilos globales: fuente Inter, colores base (modo oscuro `#242424` y claro `#ffffff`), estilos de botones, links, y centrado del `body` con `display: flex`.
- `App.css` → Estilos que **no se usan** realmente en este proyecto (vienen de la plantilla Vite: `.logo`, `.card`, animaciones). `App.jsx` no importa `App.css`, así que este archivo está huérfano. Se puede borrar o ignorar.

6.11 eslint.config.js — El corrector

Define reglas para que el código sea consistente. Usa plugins de React y React Hooks. Ignora la carpeta `dist` (donde va el build). Si escribes `let x =` sin usar, ESLint te avisa.

7. Conceptos clave

7.1 Componente

Una función que devuelve JSX. Es como una pieza de LEGO con forma propia. Ej: `PersonasList` es un componente que sabe dibujar una tarjeta.

7.2 JSX

HTML dentro de JS. Ejemplo: `<h1>Hola {nombre}</h1>`. Las llaves `{}` permiten meter variables JS dentro del HTML.

7.3 Props

Son los **parámetros** que un padre le pasa a un hijo. En `<PersonasFiltro busqueda={busqueda} />`, `busqueda` es una prop. Es como pasarle una nota al hijo.

7.4 Estado (useState)

Memoria del componente. Si el estado cambia, React vuelve a pintar. `const [contador, setContador] = useState(0)` → `contador` vale 0, `setContador(1)` lo cambia a 1 y la pantalla se actualiza.

7.5 Hook useEffect

"Código que se ejecuta en ciertos momentos". Con `[]` al final: solo al aparecer. Con `[busqueda]`: cada vez que `busqueda` cambie. Aquí se usa para traer datos una vez.

7.6 fetch + async/await

`fetch` hace una petición HTTP (como entrar a una URL). Es **asíncrono** porque Internet tarda. `await` significa "espera la respuesta sin bloquear". `try/catch/finally` maneja éxito/error/siempre.

7.7 map y filter

- `map`: Transforma cada elemento. `[1, 2, 3].map(x => x*2) → [2, 4, 6]`. Aquí se usa para agregar `Seleccionado` y para crear componentes.
- `filter`: Filtra. `[1, 2, 3].filter(x => x>1) → [2, 3]`. Aquí filtra personas por búsqueda y selección.

7.8 Spread ...

`...persona` copia todas las propiedades de `persona` en un nuevo objeto. Es como fotocopiar.

7.9 Operador ternario ? :

`condicion ? valorSiTrue : valorSiFalse`. Ejemplo: `isSelected ? 'verde' : 'gris'`.

8. Flujo completo de datos

1. Usuario abre `http://localhost:5173`
2. Navegador carga `index.html` -> `div#root` vacío + script `main.jsx`
3. `main.jsx` renderiza `<App />` dentro de `#root`
4. `App.jsx` renderiza `<header>` + `<Personas />`
5. `Personas.jsx` se monta:
 - a) Inicializa estados: `personas=[], cargando=true, busqueda="", mostrarSeleccionados=false`
 - b) `useEffect` se dispara -> `fetch` a `generate-random.org`
 - c) Mientras espera, muestra "Cargando datos desde la API..."
6. API responde con 50 personas
7. Código hace `map` agregado `Seleccionado:"n", setPersonas(...), setCargando(false)`
8. React re-renderiza: ahora muestra `<PersonasFiltro>` + `<PersonasGrid>` con 50 tarjetas
9. Usuario escribe "ana" en input:
 - > `onChange` -> `setBusqueda("ana")` -> `Personas.jsx` re-renderiza
 - > `filter` recalcula `personasAMostrar` -> solo las que contienen "ana"
 - > `PersonasGrid` recibe lista filtrada -> muestra menos tarjetas
10. Usuario hace clic en tarjeta con `ssn "123..."`:
 - > `onClick` -> `onToggle` -> `toggleSeleccion("123...")`
 - > `map` invierte `Seleccionado` de esa persona -> `setPersonas`
 - > tarjeta se vuelve borde verde
11. Usuario marca checkbox "Mostrar solo seleccionados":
 - > `setMostrarSeleccionados(true)` -> `filter` ahora exige `Seleccionado==="s"`
 - > solo se ven las verdes

Diagrama de props (quién le pasa qué a quién):

```

Personas (padre cerebro)
  ■■■ estado: personas, busqueda, mostrarSeleccionados, cargando
  ■■■ función: toggleSeleccion
  ■
  ■■■> PersonasFiltro (hijo)
  ■   recibe: busqueda, setBusqueda, mostrarSeleccionados, setMostrarSeleccionados
  ■   no tiene estado propio, solo inputs controlados
  ■
  ■■■> PersonasGrid (hijo)
  ■   recibe: personas (ya filtradas) + onToggle
  ■
  ■■■> PersonasList (nieto, x50)
  ■   recibe: persona (objeto) + onToggle (para esa persona)
  ■   muestra tarjeta, onClick invierte selección
  
```

9. Glosario

Término	Significado simple
API	Un mesero que te trae datos de otro servidor. Aquí: <code>generate-random.org</code> .
SSN	Social Security Number, DNI de EE.UU. Aquí se usa como ID único.
CORS	Permiso del servidor para que otro sitio le pida datos. Si falla, ves error en consola.
Props drilling	Pasar props de abuelo a nieto pasando por el padre (<code>Personas</code> -> <code>Grid</code> -> <code>List</code>).
Renderizar	Dibujar en pantalla. React lo hace cada vez que cambia el estado.
Estado controlado	Un input cuyo valor viene del estado de React (<code>value={busqueda}</code>).
Desestructuración	<code>function Comp({a, b})</code> es atajo para <code>props.a</code> , <code>props.b</code> .
Módulo	Archivo JS que puede importar/exportar cosas.
SWC	Traductor rápido de JSX a JS.
ESLint	Linter/corrector.

10. Errores comunes y preguntas

P: Escribí algo y la pantalla quedó en blanco. ¿Qué hago? R: Abre la consola del navegador (F12 → Console). React muestra errores en rojo. Revisa que hayas cerrado todas las llaves y etiquetas.

P: ¿Por qué useState importado en App.jsx no se usa? R: Es código sobrante. Puedes borrar `import { useState }` sin problema.

P: ¿Por qué la API a veces no responde o da error CORS? R: Es un servicio gratuito externo. Si cae, la app queda en "Cargando..." o muestra error en consola. Para pruebas, puedes reemplazar el fetch por datos locales falsos.

P: ¿Qué pasa si dos personas tienen el mismo SSN? R: El key se duplicaría y React se confundiría. Con esta API es improbable, pero en producción se usaría un ID único real.

P: ¿Dónde está el CSS de text_rojo y card--selected? R: No existe en `index.css` ni `App.css`. El efecto visual real viene del `style` en línea (borde verde). Esas clases no hacen nada hoy; se podrían definir.

P: ¿Cómo cambio el número de personas de 50 a 10? R: En `Personas.jsx` línea 14, cambia `count=50` por `count=10`.

P: ¿Cómo agrego un orden alfabético? R: Después de filtrar, agrega `.sort((a,b) => a.last_name.localeCompare(b.last_name))`.

11. Qué se podría mejorar

- 1 **Nombres:** Renombrar `PersonasList.jsx` a `PersonaCard.jsx` para evitar confusión.
- 2 **Tipado:** Usar `PropTypes` o `TypeScript` para validar props.
- 3 **Manejo de error visible:** Mostrar en pantalla "Error al cargar" en lugar de solo `console.error`.
- 4 **Loading skeleton:** En vez de texto "Cargando...", mostrar tarjetas grises animadas.
- 5 **Persistencia:** Guardar seleccionados en `localStorage` para que no se borren al recargar.
- 6 **Paginación:** Si fueran 1000 personas, no mostrar todo a la vez.
- 7 **CSS:** Mover estilos en línea a clases CSS para mantener orden.
- 8 **Accesibilidad:** Agregar `aria-label` a inputs y tarjetas.

Conclusión

Este proyecto es un **CRUD de lectura + selección local** minimalista pero completo para aprender React:

- **Trae datos** (`fetch` + `useEffect`)
- **Guarda estado** (`useState`)
- **Filtra** (`filter` + `includes`)
- **Comunica componentes** (props y callbacks)
- **Reacciona a eventos** (`onChange`, `onClick`)

Si entendiste este documento, ya entiendes el 70% de cualquier app React básica. El siguiente paso es modificar algo: cambia el placeholder, agrega un campo (ej: teléfono), cambia el color del borde, y ve cómo reacciona la app.

Autor del documento: *Generado automáticamente mediante análisis del código fuente.*

Fecha: *Septiembre 2026*

Proyecto: *2- Personas / Personas - Vite + React 18*